

数据结构

时间复杂度

$O(1)$ ：常数次操作。例：数组按下标访问 $a[i]$ ；栈入栈/出栈；队列入队/出队。

$O(n)$ ：遍历一次。例：顺序表/链表遍历找最大值；顺序查找某个值 x 。

$O(\log n)$ ：每次把问题规模减半。例：二分查找。

$O(n \log n)$ ：分治 + 每层处理 n 个元素。例：归并排序；快速排序平均情况。

$O(n^2)$ ：双重循环/两两比较。例：冒泡排序、选择排序、插入排序（最坏）。

$O(2^n)$ / $O(n!)$ ：暴力枚举，增长极快。例：枚举所有子集；枚举全排列。

空间复杂度

$O(1)$ ：只用常数个临时变量。例：原地遍历数组找最大值；顺序表删除元素用覆盖前移；迭代求和。

$O(n)$ ：额外开了长度为 n 的数组/表/栈/队列，或递归深度为 n 。例：归并排序需要辅助数组；用栈保存 n 个元素；递归遍历链表（递归栈约 n 层）。

$O(\log n)$ ：递归深度约为 $\log n$ 。例：递归版二分查找；快速排序平均递归栈（最坏可能退化到 $O(n)$ ）。

单链表

1) 头插法建表 (Head Insert)

把新结点插到头结点后面（会逆序）

```
s->next = head->next;
head->next = s;
```

解释：先让 s 指向原第一个结点，再让 $head$ 指向 s 。

2) 尾插法建表 (Tail Insert)

把新结点接到链表尾部（保持顺序，需 $tail$ ）

```
s->next = NULL;
tail->next = s;    让原来的尾结点指向新结点
tail = s;          新的尾结点就是 s  更新尾节点
```

解释： $tail$ 始终指向最后一个结点，尾插是 $O(1)$ 。

3) 插入元素（在结点 p 后插入 s ）

常用于“在第 i 个位置插入”：先找到前驱 p （第 $i-1$ 个结点）

```
s->next = p->next;
p->next = s;
```

解释：先接后面再接前面，避免断链。

4) 删除元素（删除 p 的后继结点）

删除第 i 个元素：先找到前驱 p，令 q = p->next

```
q = p->next;
p->next = q->next;
free(q);
```

解释：让 p 跨过 q 指向 q 的后继，再释放 q。

5) 销毁单链表（Destroy）

A. 只释放数据结点，保留头结点

```
p = head->next;
while (p) {
    q = p->next;
    free(p);
    p = q;
}
head->next = NULL;
```

B. 连头结点一起释放（head 置空）

```
p = head;
while (p) {
    q = p->next;
    free(p);
    p = q;
}
head = NULL;
```

记忆口诀

- 插入：s 先指后，再让 p 指 s
- 删除：p 跨过 q，q 释放掉

题6：以L为头结点指针，给出单链表、双链表、循环单链表和循环双链表中，指针p所指结点为尾结点的条件。

单链表p->next == NULL

双链表p->next == NULL 末结点的后继为空

循环单链表 $p \rightarrow next == L$ 尾结点的后继回到头结点

循环双链表 $p \rightarrow next == L \ \&\& \ L \rightarrow prior == p$

插入

先处理插入的节点 然后再把原来的节点连上他 否则先动原来的节点会丢失后面的节点

双链表

- 在结点 p 之后插入新结点 s (后插)

```
s->next = p->next;
s->prior = p;
if (p->next) p->next->prior = s;
p->next = s;
```

说明：先连上 s 的两头，再修补原来 $p \rightarrow next$ 的 $prior$ ，最后让 p 指向 s 。

- 在结点 p 之前插入新结点 s (前插)

```
s->prior = p->prior;
s->next = p;
if (p->prior) p->prior->next = s;
p->prior = s;
```

说明：对称操作，把 s 塞到 $p \rightarrow prior$ 和 p 中间。

-
- 删除结点 q (q 已知)

```
if (q->prior) q->prior->next = q->next;    前的后指向后
if (q->next) q->next->prior = q->prior;    后的前指向前
free(q);
```

说明：让前驱跨过 q 指向后继，让后继跨过 q 指向前驱，再释放 q 。

-
- 一个简单记忆法

- 插入：改 **4 条指针** (s 的 2 条 + 两侧结点各 1 条)
- 删除：改 **2 条指针** (前驱的 `next` + 后继的 `prior`) 再 `free`

- 双链表头插法 (带头结点 `head`，把新结点 s 插到第一个结点前面，也就是头结点后面)

```
s->next = head->next;
s->prior = head;
if (head->next) head->next->prior = s; // 原首结点的 prior 指向 s
head->next = s;
```

说明：新结点要同时接好两端（prior/next），并把原首结点的 prior 改成指向 s。

- 双链表尾插法（带头结点 head，维护尾指针 tail，把新结点 s 插到尾部）

```
s->next = NULL;
s->prior = tail;
tail->next = s;
tail = s;
```

说明：尾插主要改 tail->next，并让 s->prior 指向旧尾，最后更新 tail。

- 如果没有维护 tail（每次都从 head 找尾结点）

```
p = head;
while (p->next) p = p->next; // 找到尾结点 p

s->next = NULL;
s->prior = p;
p->next = s;
```

说明：这样尾插会多一个遍历，建表会变慢。

栈

顺序栈

（用数组实现栈）：data[MaxSize] 存元素，top 表示栈顶下标（指向栈顶元素）。

初始化（空栈）：top = -1。

栈空判定：top == -1。

栈满判定：top == MaxSize - 1。

入栈（Push）基本操作：先判断不满，再 top++，把元素放到 data[top]。

- 口诀：top++，入栈放 top 处

出栈 (Pop) 基本操作：先判断不空，取 `data[top]`，再 `top--`。

- 口诀：取 `top`，`top--`

时间复杂度：入栈/出栈/取栈顶 都是 $O(1)$ 。

链栈

用链表实现栈，一般用**带头结点的单链表**；`S` 指向头结点（不放任何数据，`S->next` 指向栈顶结点）。

栈顶/栈底：靠近头结点的一端是**栈顶**（便于 $O(1)$ 操作）；链尾方向是**栈底**。

栈空条件：`S->next == NULL`。

栈满条件：通常**不考虑**（只要还能申请到内存就能继续进栈；真正满是“内存不足”）。

进栈 (Push)：相当于**头插法**，把新结点插到头结点后面（成为新的栈顶），指针操作：

- `p->next = S->next;`
- `S->next = p;`

```
S -> p -> A -> B -> ...
      ↑
    新栈顶
```

出栈 (Pop)：删除头结点后的第一个结点（栈顶结点），指针操作：

- `p = S->next;` A
- `S->next = p->next;` B 跳过A
- 取出 `p->data`，`free(p)`

```
S -> A -> B -> ...
      ↑
    栈顶
```

时间复杂度：进栈/出栈/取栈顶 都是 $O(1)$ ；空间随元素个数增长（每个结点多一个指针域）。

迷宫问题（栈求解）

```
/*
 * 用“栈(非递归DFS)+回溯”求迷宫从入口(xi,yi)到出口(xe,ye)的一条路径
 * 约定：
 *   mg[i][j] = 1 表示墙/不可走
 *   mg[i][j] = 0 表示可走
 *   mg[i][j] = -1 表示已走过(访问标记)
 *
 * 栈中每个元素 Box 保存：
 *   i, j : 当前格子坐标
 *   di   : 该格子“上一次尝试过的方向”(用于回溯后继续试其他方向)
 *
 * 方向编码(与课件一致的常见写法):
 *   0:上  1:右  2:下  3:左
```

```

*/

#include <stdio.h>
#include <stdbool.h>

#define ROWS 6
#define COLS 6
#define MAXSIZE 100

typedef struct {
    int i, j;    // 坐标
    int di;      // 上次尝试方向, 初始为 -1
} Box;

/* 简单顺序栈 */
typedef struct {
    Box data[MAXSIZE];
    int top;
} Stack;

static void InitStack(Stack *st) { st->top = -1; }
static bool StackEmpty(Stack *st) { return st->top == -1; }
static bool Push(Stack *st, Box e) {
    if (st->top >= MAXSIZE - 1) return false;
    st->data[++st->top] = e;
    return true;
}

static bool Pop(Stack *st, Box *e) {
    if (StackEmpty(st)) return false;
    *e = st->data[st->top--];
    return true;
}

static bool GetTop(Stack *st, Box *e) {
    if (StackEmpty(st)) return false;
    *e = st->data[st->top];
    return true;
}

/* 打印当前栈路径（从栈底到栈顶） */
static void PrintPath(Stack *st) {
    printf("找到一条路径(从入口到出口): \n");
    for (int k = 0; k <= st->top; k++) {
        printf("(%d,%d) ", st->data[k].i, st->data[k].j);
        if ((k + 1) % 8 == 0) printf("\n");
    }
    printf("\n");
}

/* 示例迷宫: 外围为墙(1), 内部0为路 */
static int mg[ROWS][COLS] = {
    {1,1,1,1,1,1},
    {1,0,0,0,1,1},
    {1,0,1,0,0,1},
    {1,0,1,0,1,1},
    {1,0,0,0,0,1},
    {1,1,1,1,1,1}
}

```

```

};

/*
 * 栈法求迷宫路径：返回是否找到路径
 * 注意：这里在“退栈”时把 mg[格子] 恢复为 0（回溯恢复环境，便于理解原理）
 */
bool mgpath_stack(int xi, int yi, int xe, int ye) {
    Stack st;
    InitStack(&st);

    // 入口入栈，di=-1 表示四个方向都还没试
    Box e = {xi, yi, -1};
    Push(&st, e);
    mg[xi][yi] = -1; // 标记已访问，避免原地绕圈

    while (!StackEmpty(&st)) {
        Box topBox;
        GetTop(&st, &topBox);

        int i = topBox.i;
        int j = topBox.j;
        int di = topBox.di;

        // 1) 到达出口：栈中就是一条路径
        if (i == xe && j == ye) {
            PrintPath(&st);
            return true;
        }

        // 2) 尝试从当前格子继续走：按 di+1 的方向继续试
        bool find = false;
        int i1 = i, j1 = j;

        while (di < 3 && !find) {
            di++; // 尝试下一个方向

            // 计算相邻格(i1,j1)
            switch (di) {
                case 0: i1 = i - 1; j1 = j; break; // 上
                case 1: i1 = i; j1 = j + 1; break; // 右
                case 2: i1 = i + 1; j1 = j; break; // 下
                case 3: i1 = i; j1 = j - 1; break; // 左
            }

            // 可走条件：mg==0（没走过且不是墙）
            if (mg[i1][j1] == 0) {
                find = true;
            }
        }

        if (find) {
            // 2.1 找到可走邻居：
            // (a) 先把“当前栈顶”的 di 更新为刚成功的方向，记录已试到哪
            st.data[st.top].di = di;

            // (b) 把新格子压栈，新格子的 di 置为 -1

```

```

        Box next = {i1, j1, -1};
        Push(&st, next);

        //      (c) 标记新格子已访问
        mg[i1][j1] = -1;
    } else {
        // 2.2 四个方向都走不通：退栈回溯
        Box out;
        Pop(&st, &out);

        // 回溯恢复环境（可选，但符合“回退后恢复环境”的原理）
        mg[out.i][out.j] = 0;
    }
}

// 栈空仍没到出口：无路
printf("无路可达出口。\\n");
return false;
}

static void PrintMaze(void) {
    printf("迷宫(1墙 0路): \\n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%d ", mg[i][j] == -1 ? 0 : mg[i][j]); // 打印时把-1当成0看更直观
        }
        printf("\\n");
    }
}

int main(void) {
    // 入口(1,1), 出口(4,4)（可按需修改）
    int xi = 1, yi = 1, xe = 4, ye = 4;

    PrintMaze();
    printf("入口=(%d,%d), 出口=(%d,%d)\\n", xi, yi, xe, ye);

    // 注意: mgpath_stack 会修改 mg（标记/回溯），如果想重复跑，请重置 mg
    mgpath_stack(xi, yi, xe, ye);

    return 0;
}

```

队列

顺序队

(用数组实现队列, MaxSize 固定): `data[MaxSize]` 存元素, 用两个指针/下标 `front`、`rear` 管理。

约定:

- `rear` 指向队尾元素 (最后一个入队的元素)
- `front` 指向队头元素的前一个位置 (不是队头本身)

初始化: `front = rear = -1` (空队)。

队空条件: `front == rear`。

入队 (EnQueue): 先判满, 再

- `rear++`
- `data[rear] = e`

出队 (DeQueue): 先判空, 再

- `front++`
- `e = data[front]`

队满条件 (非循环顺序队): `rear == MaxSize - 1`。

- 注意: 这种顺序队即使前面出队空出位置, 只要 `rear` 到末尾也**不能再入队** (会产生“假溢出”问题)。

时间复杂度: 入队/出队都是 **O(1)** (只移动指针并存取一个元素)。

假溢出 (也叫“伪溢出”) 指的是: 顺序队列 (非循环队列) 里, 数组末尾已经被 `rear` 走到头, 虽然前面因为出队空出了位置, 但仍然判定为“队满”, 无法继续入队。

- 为什么会发生
顺序队列用数组存储, `rear` 只增不减 (不回头), 当
 - 队满条件: `rear == MaxSize - 1`
即使此时 `front` 已经因为出队变大, 前面出现空位, `rear` 也不能利用这些空位, 所以出现“看起来满了”的现象。
- 解决办法 (常考)
 1. **循环队列**: 让 `front/rear` “取模回绕”, 空位可复用 (最常用)。
 2. **元素整体搬移**: 把现有元素往前挪到数组前端, 再继续入队 (效率较低)。
 3. 改用 **链队列**: 链式存储不依赖连续数组, 也就没有假溢出 (内存够就能入队)。

环形队列

(循环队列)：用数组实现，但把数组首尾“接起来”，`front/rear` 通过取模 `%MaxSize` 循环移动，用来解决顺序队列的假溢出。

- **方案一**：只用 `front`、`rear` (牺牲一个存储单元区分空/满)
 - 队空条件：`front == rear`
 - 队满条件：`(rear + 1) % MaxSize == front` (再进一格就追上 `front` 了)
 - 入队：`rear = (rear + 1) % MaxSize; data[rear] = e;`
 - 出队：`front = (front + 1) % MaxSize; e = data[front];`
 - 特点：最多只能存 **MaxSize-1** 个元素 (少用 1 格来避免空满混淆)。
- 环形队列常用计数公式 (仍用 `front`、`rear` 的版本)：
 - 队中元素个数：`count = (rear - front + MaxSize) % MaxSize`
 - 已知 `front`、`count` 求 `rear`：`rear = (front + count) % MaxSize`
 - 已知 `rear`、`count` 求 `front`：`front = (rear - count + MaxSize) % MaxSize`
- **方案二**：增加计数器 `count` (不牺牲单元，可放满 `MaxSize` 个)
 - 结构体常见：`data[MaxSize] + front + count` (`rear` 可由 `front/count` 推出)
 - 队空：`count == 0`
 - 队满：`count == MaxSize`
 - 入队/出队仍然按取模移动 `front/rear`，同时维护 `count++ / count--`
 - 特点：最多可存 **MaxSize** 个元素 (但多维护一个变量)。

链队

用链表实现队列 (先进先出 FIFO)，这里用的是**不带头结点的单链表**。

- 两个关键指针：
 - `front` (队头指针)：指向**第一个结点/队头元素 a1**
 - `rear` (队尾指针)：指向**最后一个结点/队尾元素 an**
- 入队 (EnQueue)：在**队尾插入**新结点 (尾插)，时间 **O(1)**
 - 空队时：`front = rear = s`
 - 非空时：`rear->next = s; rear = s;` (并令 `s->next = NULL`)
- 出队 (DeQueue)：在**队头删除**结点 (删除首结点)，时间 **O(1)**
 - `p = front; front = front->next; free(p);`
 - 若删除后 `front == NULL`，说明队空，要把 `rear` 也置空：`rear = NULL`
- 队空条件：`front == NULL` (通常也等价于 `rear == NULL`)。

- 栈满：一般不考虑（只受内存限制）。

带头结点的单链表。

- 初始化：
 - `front = rear = 头结点`，且 `front->next = NULL`
- 队空条件：
 - `front == rear`（或 `front->next == NULL`）
- 入队（EnQueue）：在队尾插入新结点（尾插）， $O(1)$
 - `s->next = NULL; rear->next = s; rear = s;`
- 出队（DeQueue）：删除头结点后的第一个结点（首元结点）， $O(1)$
 - `p = front->next; front->next = p->next;`
 - 若 `rear == p`（删到最后一个），则 `rear = front`
 - 取 `p->data`，`free(p)`
- 队满：一般不考虑（只受内存限制）

结构	存储方式	关键指针含义	判空	判满	入/出操作时间复杂度	备注
顺序栈	数组	<code>top</code> 指向栈顶元素下标	<code>top == -1</code>	<code>top == MaxSize-1</code>	Push/Pop/GetTop 都 $O(1)$	容量固定
链栈	带头单链表	<code>s</code> 指头结点； <code>s->next</code> 是栈顶	<code>s->next == NULL</code>	一般不考虑（内存不足才算满）	Push/Pop/GetTop 都 $O(1)$	Push=头插，Pop=头删
顺序队（非循环）	数组	<code>rear</code> 指队尾元素； <code>front</code> 指队头前一格	<code>front == rear</code>	<code>rear == MaxSize-1</code>	En/De 都 $O(1)$	有假溢出
循环队（方案一）	数组+取模	只用 <code>front/rear</code> （仍可按“front 前一格”约定）	<code>front == rear</code>	<code>(rear+1)%MaxSize == front</code>	En/De 都 $O(1)$	最多存 $MaxSize-1$
循环队（方案二）	数组+取模	<code>head/tail + count(cnt)</code> 计数	<code>count==0</code>	<code>count==MaxSize</code>	En/De 都 $O(1)$	可存满 $MaxSize$
链队（不带头）	单链表	<code>front</code> 指首元； <code>rear</code> 指尾元	<code>front == NULL</code>	一般不考虑（内存不足）	En/De 都 $O(1)$	出队删首元；空队入队要 <code>front=rear</code>

结构	存储方式	关键指针含义	判空	判满	入/出操作时间复杂度	备注
链队 (带头)	带头单链表	front 指头结点; rear 指尾结点	front==rear 或 front->next==NULL	一般不考虑(内存不足)	En/De 都 O(1)	删到最后一个后要 rear=front

11. 设从键盘输入一序列的字符 a_1 、 a_2 、...、 a_n 。设计一个算法实现这样的功能：若 a_i 为数字字符， a_i 进队，若 a_i 为小写字母时，将队首元素出队，若 a_i 为其他字符，表示输入结束。要求使用环形队列。

```
int main(void)
{
    char q[CAP];
    int head = 0, tail = 0, cnt = 0; // 环形队列
    int ch;

    while ((ch = getchar()) != EOF) {
        if (ch >= '0' && ch <= '9') {
            if (cnt == CAP) {
                puts("队列满,不能进队");
            } else {
                q[tail] = (char)ch;
                tail = (tail + 1) % CAP;
                ++cnt;
            }
        } else if (ch >= 'a' && ch <= 'z') {
            if (cnt == 0) {
                puts("队列空,不能出队");
            } else {
                char e = q[head];
                head = (head + 1) % CAP;
                --cnt;
                printf("出队元素:%c\n", e);
            }
        } else {
            break;
        }
    }
    return 0;
}
```

迷宫问题 (队列求解)

```
/*
 * 用队列(BFS)求迷宫路径: 入口(xi,yi) -> 出口(xe,ye)
 * 尽量按课件思路写:
```

```

*   - 队列用顺序队（front 初值=-1, rear 初值=-1）
*   - 出队: front++, 取 data[front], 此时 front 正好是“刚出队结点下标”
*   - 新结点入队时: pre = qu->front (把刚出队的结点当作前驱)
*   - 走过的格子标记为 -1, 避免重复入队
*
* 约定:
*   mg[i][j] = 1  墙/不可走
*   mg[i][j] = 0  路/可走
*   mg[i][j] = -1 已访问
*
* BFS 特点: 第一次到达出口时得到的是“步数最短路径”（每步代价相同的情况下）
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define ROWS 6
#define COLS 6
#define QMAX (ROWS * COLS + 10)

typedef struct {
    int i, j;    // 坐标
    int pre;     // 前驱结点在队列数组中的下标（用于回溯输出路径）
} Box;

typedef struct {
    Box data[QMAX];
    int front;   // 指向队头元素的“前一位置”（课件风格）
    int rear;    // 指向队尾元素
} QuType;

/* ===== 顺序队基本操作（与课件一致的 front/rear 语义） ===== */
static QuType* InitQueue(void) {
    QuType *qu = (QuType*)malloc(sizeof(QuType));
    if (!qu) return NULL;
    qu->front = qu->rear = -1;
    return qu;
}

static bool QueueEmpty(QuType *qu) {
    return qu->front == qu->rear;
}

static void DestroyQueue(QuType *qu) {
    free(qu);
}

static bool enqueue(QuType *qu, Box e) {
    if (qu->rear >= QMAX - 1) return false; // 队满（本例容量足够就行）
    qu->data[++qu->rear] = e;
    return true;
}

static bool dequeue(QuType *qu, Box *e) {
    if (QueueEmpty(qu)) return false;
    *e = qu->data[++qu->front]; // 注意: front++ 后 front 就是“刚出队元素下标”
    return true;
}

```

```

/* ===== 输出路径：从“某个结点下标 idx”沿 pre 回溯到 -1 ===== */
static void printPath(QuType *qu, int idx) {
    int path[QMAX];
    int k = 0;

    // 沿 pre 回溯：出口 -> ... -> 入口
    while (idx != -1 && k < QMAX) {
        path[k++] = idx;
        idx = qu->data[idx].pre;
    }

    // 逆序输出：入口 -> ... -> 出口
    printf("找到一条路径（入口->出口）：\n");
    for (int t = k - 1; t >= 0; t--) {
        int p = path[t];
        printf("(%d,%d) ", qu->data[p].i, qu->data[p].j);
        if ((k - t) % 8 == 0) printf("\n");
    }
    printf("\n");
}

/* ===== 示例迷宫（外围是墙，避免越界判断也能运行） ===== */
static int mg[ROWS][COLS] = {
    {1,1,1,1,1,1},
    {1,0,0,0,1,1},
    {1,0,1,0,0,1},
    {1,0,1,0,1,1},
    {1,0,0,0,0,1},
    {1,1,1,1,1,1}
};

/*
 * BFS(队列)找路径：与课件 mgpath1 思路一致
 * 返回：找到路径 true；否则 false
 */
bool mgpath_queue(int xi, int yi, int xe, int ye) {
    Box e;
    int i, j, di, i1, j1;

    QuType *qu = InitQueue();
    if (!qu) {
        printf("队列初始化失败（内存不足）\n");
        return false;
    }

    // 入口入队，pre=-1
    e.i = xi; e.j = yi; e.pre = -1;
    enqueue(qu, e);

    mg[xi][yi] = -1; // 标记已访问

    while (!QueueEmpty(qu)) {
        // 出队一个方块 e（当前扩展点）
        dequeue(qu, &e);
        i = e.i;
        j = e.j;

```

```

// 找到出口：此时 qu->front 就是“刚出队结点下标”，可用于回溯打印
if (i == xe && j == ye) {
    printPath(qu, qu->front);
    DestroyQueue(qu);
    return true;
}

// 扫描四个方向（与课件 switch(di) 一致：上、右、下、左）
for (di = 0; di < 4; di++) {
    switch (di) {
        case 0: i1 = i - 1; j1 = j; break; // 上
        case 1: i1 = i; j1 = j + 1; break; // 右
        case 2: i1 = i + 1; j1 = j; break; // 下
        default: i1 = i; j1 = j - 1; break; // 左
    }

    // 可走且未访问：mg==0
    if (mg[i1][j1] == 0) {
        Box ne;
        ne.i = i1;
        ne.j = j1;
        ne.pre = qu->front; // 关键：前驱就是“刚出队”的那个结点下标
        enqueue(qu, ne);
        mg[i1][j1] = -1; // 立刻标记，避免重复入队
    }
}

// 队列空还没到出口：无路
printf("无路可达出口。\\n");
DestroyQueue(qu);
return false;
}

int main(void) {
    int xi = 1, yi = 1;
    int xe = 4, ye = 4;

    printf("入口=(%d,%d)，出口=(%d,%d)\\n", xi, yi, xe, ye);
    mgpath_queue(xi, yi, xe, ye);

    return 0;
}

```

本质不同

- **用栈**：深度优先搜索（DFS）——沿一条路一直走，走不通再回溯。
- **用队列**：广度优先搜索（BFS）——按“距离入口的层数”一层层扩展。

得到的路径性质

- **队列(BFS)**: 第一次到达出口时得到的通常是**最短路径 (步数最少)**。
- **栈(DFS)**: 找到的是**某一条可行路径**, 不保证最短, 可能绕远。

访问/标记策略差异

- **BFS**: 一般“入队即标记已访问”, 每个格子最多入队一次, 避免重复。
- **DFS**: 常见做法是“入栈标记”, 回溯时可能需要恢复 (或不恢复但会影响能否走其他分支), 实现上更容易出现细节差别。

空间与搜索形态

- **BFS**: 队列里可能同时存很多“同层结点”, 在分支多时空间可能更大。
- **DFS**: 栈主要保存当前路径, 通常更像“一条线”往深处走, 空间常更接近“路径长度”。

使用场景

- 想要**最短路**: 用队列 (BFS)。
- 只要**任意一条路**、实现简单或内存更紧张: 用栈 (DFS)。

串

`StrAssign(&s, cstr)`: **串赋值**。把字符串常量/字符数组 `cstr` 赋给串 `s`。

`StrCopy(&s, t)`: **串复制**。把串 `t` 复制给串 `s` (令 `s=t`)。

`StrEqual(s, t)`: **判等**。相等返回真, 否则返回假。

`StrLength(s)`: **求串长**。返回串 `s` 的字符个数。

`Concat(s, t)`: **串连接**。返回新串 = `s` 后面接 `t`。

`SubStr(s, i, j)`: **求子串**。从 `s` 的第 `i` 个字符开始 (通常从 1 计数), 取连续 `j` 个字符组成子串并返回。

`InsStr(s1, i, s2)`: **插入子串**。把 `s2` 插入到 `s1` 的第 `i` 个字符位置 ($1 \leq i \leq n+1$) 并返回新串。

`DelStr(s, i, m)`: **删除子串**。从第 `i` 个字符开始删除长度为 `m` 的子串, 返回新串。

`RepStr(s, i, j, t)`: **替换**。把 `s` 中从第 `i` 个字符开始长度为 `j` 的子串, 用串 `t` 替换, 返回新串。

`DispStr(s)`: **输出串**。输出串 `s` 的所有字符。

BF (Brute-Force) 算法=暴力/简单匹配算法: 用来做**串的模式匹配** (在主串 `s` 中查找模式串 `t` 第一次出现的位置)。

基本思路: 从 `s` 的每一个可能起始位置开始, 把 `t` 的字符依次与 `s` 对齐比较; 不匹配就把 `t` 右移一位, 重新比较。

两个指针/下标常见含义：

- i 指向主串 s 当前比较位置
- j 指向模式串 t 当前比较位置
- 若 $s[i] == t[j]$: $i++$, $j++$ (继续比较下一对字符)
- 若不相等：需要回溯
 - $i = i - j + 1$ (回到本次匹配起点的下一位)
 - $j = 0$ (或 $j = 1$, 看你们下标从 0 还是从 1 计)

匹配成功条件： $j == m$ (或 $j > m$) , 说明模式串 t 的 m 个字符全匹配上了。

匹配失败条件： i 已经走到主串末尾仍未成功 (通常 $i > n-m$ 后不可能再匹配) 。

时间复杂度 (主串长度 n , 模式串长度 m) :

- **最坏：** $O(n*m)$ (大量回溯反复比较)
- **平均** (你图里的选择题标准答案) : $O(n*m)$
- **最好：** $O(m)$ (一开始就匹配成功) 或 $O(n)$ (很快发现不匹配, 视实现而定)

BF 的缺点/特点：

- 实现简单, 但回溯多、效率低; 后续常用 **KMP** 来减少回溯、提高匹配效率。

BMP算法

计算next

0开头和1开头不同

-1 0

11. 采用顺序结构存储串, 设计一个算法求串 s 中出现的第一个最长重复子串的下标和 长度。

```
void fun(SqString s, SqString &t) {
    int startIndex = 0, maxLen = 0, currentLen, i, j, k;
    i = 0;

    while (i < s.length) { // 从 i 开始查找
        j = i + 1; // 从 i 后的字符开始找重复子串
        while (j < s.length) {
            if (s.data[i] == s.data[j]) { // 找到重复子串
                currentLen = 1; // 当前重复子串的长度为 1
                // 比较后续字符
                for (k = 1; s.data[i + k] == s.data[j + k] && (i + k < s.length
&& j + k < s.length); k++) {
                    currentLen++;
                }
                if (currentLen > maxLen) {
                    startIndex = i;
                    maxLen = currentLen;
                }
            }
            j++;
        }
        i++;
    }
}
```

```
        }
        j += currentLen;
    } else {
        j++; // 继续查找
    }
}
i++; // 扫描下一个字符
}

t.length = maxLen;
for (i = 0; i < maxLen; i++) {
    t.data[i] = s.data[startIndex + i];
}
}
```

算法	预处理时间	匹配时间 (最好)	匹配时间 (最坏)	总时间 (最坏)	额外空间	关键特点
BF (朴素/暴力)	O(1)	O(m) (一开始就匹配成功)	O(n·m) (大量回溯重复比较) (思否)	O(n·m) (思否)	O(1)	实现最简单, 但可能反复回退
KMP / “BMP” (next/nextval)	O(m) (算 next/失配函数) (知乎专栏)	O(n)	O(n) (匹配阶段线性) (GeeksforGeeks)	O(n+m) (GeeksforGeeks)	O(m)	失败时不回退主串 <code>i</code> , 靠 next 跳 <code>j</code>

递归

递归 (Recursion)：函数/过程在定义中**直接或间接调用自身**，用“把大问题化成小规模同类问题”来求解。

递归模型 (递归结构)：递归算法的抽象表达，通常由两部分组成：

- **递归出口 (边界条件)**：递归到什么时候结束（否则会无限递归）。

- **递归体（递推关系）**：把规模为 n 的问题转换为规模更小的问题 ($n-1$ 、 $n/2$...)。
- 例（阶乘）：
 - 出口：`fun(1)=1`
 - 递归体：`fun(n)=n*fun(n-1), n>1`

尾递归（Tail Recursion）：递归调用语句是函数中**最后一条执行语句**（调用后不再做额外计算）。

- 典型形式：`return f(n-1, ...)`
- **优点/意义**：尾递归更容易改写成等价的**循环**（减少栈开销）。
- 注意：`return n * fun(n-1)` **不是尾递归**（递归返回后还要乘 n ）。

递归与栈的关系：

- 一般递归在运行时需要系统调用栈保存现场（参数、返回地址、局部变量）。
- **尾递归**更容易被循环替代；**非尾递归**通常可用“显式栈”改写成非递归算法。

递归数据结构：数据结构的定义中包含**自身类型**（或指向自身类型的指针/子结构）。

- 例：单链表结点
 - `struct LNode { ElemType data; struct LNode *next; }`
 - `next` 指向同类型结点 $\Rightarrow$ 链表具有递归结构（链表 = 结点 + 后续链表/空）。

```
/*
 * 递归求迷宫路径（DFS + 回溯），基于你图里的 mgpath 模型：
 * ① 将 (xi, yi) 加入 path
 * ② 置 mg[xi][yi] = -1（标记已走过）
 * ③ 对四周可走相邻方块递归 mgpath(...)
 * ④ 回退一步：path.length-- 并置 mg[xi][yi] = 0（取消标记，找所有解）
 *
 * 说明：
 * - mg[][]: 0 表示可走，1 表示墙，-1 表示本次路径已访问（避免走回头路）
 * - 这里用“外圈全是墙”的迷宫，避免越界判断（你也可以改为显式判断边界）
 */

#include <stdio.h>

#define ROW 8
#define COL 8
#define MAXSIZE (ROW * COL)

typedef struct {
    int i; // 行号
    int j; // 列号
} Box;

typedef struct {
    Box data[MAXSIZE];
    int length; // 路径长度
} PathType;

/* 迷宫：外圈为墙(1)，内圈 0 可走，1 墙 */
int mg[ROW][COL] = {
```

```

{1,1,1,1,1,1,1,1},
{1,0,0,0,1,0,0,1},
{1,0,1,0,1,0,1,1},
{1,0,1,0,0,0,0,1},
{1,0,1,1,1,1,0,1},
{1,0,0,0,0,0,0,1},
{1,1,0,1,0,1,0,1},
{1,1,1,1,1,1,1,1}
};

static int count = 0;

/* 输出一条路径（和你图里一样：每 5 个方块换行） */
static void PrintPath(const PathType *path) {
    int k;
    printf("迷宫路径%d如下: \n", ++count);
    for (k = 0; k < path->length; ++k) {
        printf("\t(%d, %d)", path->data[k].i, path->data[k].j);
        if ((k + 1) % 5 == 0) printf("\n");
    }
    printf("\n\n");
}

/* 递归找所有路径：从 (xi,yi) 到 (xe,ye) */
void mgpath(int xi, int yi, int xe, int ye, PathType *path) {
    /* 递归出口：到达终点 */
    if (xi == xe && yi == ye) {
        path->data[path->length].i = xi;
        path->data[path->length].j = yi;
        path->length++;

        PrintPath(path);

        /* 回溯：弹出终点，继续找其它解 */
        path->length--;
        return;
    }

    /* 若当前位置不可走（墙或已访问），直接返回 */
    if (mg[xi][yi] != 0) return;

    /* ① 将 (xi, yi) 添加到 path 中 */
    path->data[path->length].i = xi;
    path->data[path->length].j = yi;
    path->length++;

    /* ② 置 mg[xi][yi] = -1（标记已走过） */
    mg[xi][yi] = -1;

    /* 四个方向：上、右、下、左 */
    const int di[4] = {-1, 0, 1, 0};
    const int dj[4] = {0, 1, 0, -1};

    /* ③ 对四周每一个可走相邻方块递归 */
    for (int k = 0; k < 4; ++k) {
        int ni = xi + di[k];

```

```

    int nj = yi + dj[k];

    /* 由于外圈是墙，这里不写边界判断也安全；可走条件是 0 */
    if (mg[ni][nj] == 0 || (ni == xe && nj == ye && mg[ni][nj] == 0)) {
        mgpath(ni, nj, xe, ye, path);
    }
}

/* ④ path 回退一步，并置 mg[xi][yi] = 0（取消标记） */
mg[xi][yi] = 0;
path->length--;
}

int main(void) {
    PathType path;
    path.length = 0;

    /* 起点、终点（按你的迷宫自行改坐标） */
    int xi = 1, yi = 1;    // 起点
    int xe = 6, ye = 6;    // 终点

    if (mg[xi][yi] != 0 || mg[xe][ye] != 0) {
        printf("起点或终点不可走! \n");
        return 0;
    }

    mgpath(xi, yi, xe, ye, &path);

    if (count == 0) {
        printf("没有找到任何路径。 \n");
    }
    return 0;
}

```

数组

一维数组是**顺序存储**：所有元素在内存中**连续**排列。

已知首元素地址 $LOC(a_1)$ ，每个元素占 k 个存储单元，则第 i 个元素地址：

- $LOC(a_i) = LOC(a_1) + (i-1) * k$
- 含义：前面有 $i-1$ 个元素，每个占 k ，所以偏移 $(i-1)k$ 。

一维数组具有**随机存取 (Random Access)**：按下标取元素可直接算地址，时间 $O(1)$ 。

二维数组 $A(m \times n)$ 仍然在内存中**按一维连续空间存放**，必须规定存储顺序：

- **行优先 (Row-major)**：先存第 1 行，再存第 2 行... (C 语言常用)

- **列优先 (Column-major)**：先存第 1 列，再存第 2 列... (Fortran/Matlab 常见)

二维数组地址计算 (假设下标从 1 开始, 首元素 $A[1][1]$ 地址为 $LOC(A[1][1])$, 每元素占 k) :

- 行优先: $LOC(A[i][j]) = LOC(A[1][1]) + ((i-1)*n + (j-1)) * k$
- 列优先: $LOC(A[i][j]) = LOC(A[1][1]) + ((j-1)*m + (i-1)) * k$

若下标从 0 开始, 把 $(i-1)$ 改成 i , $(j-1)$ 改成 j 即可。

10. 设计一个算法, 计算一个三元组表表示的稀疏矩阵的对角线元素之和。

```
bool fun(TSMatrix mat, ElemType &sum) {
    sum = 0;
    if (mat.rows != mat.cols) { // 如果行数不等于列数, 返回 false
        printf("矩阵不是方阵\n");
        return false;
    }
    for (int i = 0; i < mat.num; i++) {
        // 如果行号等于列号, 则是对角线元素
        if (mat.data[i].row == mat.data[i].col) {
            sum += mat.data[i].value; // 累加对角线元素的值
        }
    }

    return true;
}
```

特殊矩阵的压缩存储: 利用矩阵里大量元素“重复/为 0/有规律”, 只存**有用部分**, 节省空间。

常见的**特殊矩阵类型** (课件列的 3 类) :

- **对称矩阵**: 满足 $a[i][j] = a[j][i]$ (关于主对角线对称) 。
 - 压缩思路: 只存上三角或下三角 (含主对角线) 即可, 另一半由对称性推出来。
- **三角矩阵**: 分为
 - **上三角矩阵**: 主对角线下方全为 0 (或同一常数), 只需存上三角部分。
 - **下三角矩阵**: 主对角线上方全为 0 (或同一常数), 只需存下三角部分。
- **对角矩阵**: 除主对角线外其余元素全为 0 (或同一常数) 。
 - 压缩思路: 只存主对角线 n 个元素即可。

共同点: 它们通常都是**方阵** (行数 = 列数 = n) , 所以压缩存储常以 $n \times n$ 为前提。

存储量 (常用结论, 便于记) :

- 对称矩阵 / 上(下)三角矩阵: 需要存的元素个数
 - $n(n+1)/2$ (存一半 + 主对角线)
- 对角矩阵: 需要存的元素个数
 - n (只存主对角线)

压缩存储的目的：

- 从 n^2 个元素的普通存储，减少到上面这些数量级，从而节省大量空间。

树

遍历

线索二叉树

满二叉树

完全二叉树

哈夫曼树

1、为第六版教材的第211页上【例7.14】设计两种新的算法 LnodeNum(b, k), 求二叉树b中第k层的结点数。要求：1) 分别采用先序遍历和层次遍历的思路来求解；2) 算法函数只有两个参数，并且返回值为结点数；3) 编写程序，验证算法的正确性，提供程序测试数据和运行结果的截图。

(1) 采用先序遍历方法求解，有两种算法，算法 1 最简单，代码如下：

```
int Lnodenum1(BTNode *b, int k) //求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);
  else //求左右子树第 k-1 层节点个数之和
    return( Lnodenum2(b->lchild,k-1) + Lnodenum2(b->rchild,k-1) );
}
```

(2) 采用先序遍历方法求解的算法 2，代码如下：

```
int Layer = 1; //全局变量 L 记录当前访问的结点层次号
void nodeNum(BTNode *b, int k, int &n) // 采用先序遍历方法 2 求解
{ if(b!=NULL)
  { if(Layer==k)
    { n++;
      Layer++;
      nodeNum(b->lchild, k, n);
      nodeNum(b->rchild, k, n);
      Layer--;
    }
  }
}

int Lnodenum2(BTNode *b,int k) // 求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);
  int n=0;
  nodeNum(b, k, n); // 调用上面的函数
  return n;
}
```

```
}
```

(3) 采用分层次的层次遍历方法求解，算法代码如下：

```
int Lndenum3(BTNode *b, int k) // 求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);

  BTNode *p, *que[MaxSize]; //定义结点指针和顺序队列
  int f=-1,r=-1; //初始化队头和队尾指针
  int lev=1; //当前层次为第 1 层
  r=(r+1)%MaxSize;
  que[r] = b; //根结点入队

  while(f!=r) //队列非空时循环
  { int num=(r-f+MaxSize)%MaxSize; //求当前层次的结点数 (即队列长度)
    if(lev==k) return num; // 当前层为第 k 层, 返回该层结点数
    for(int i=0; i<num; i++)
    { f=(f+1)%MaxSize;
      p = que[f]; //出队结点 p  if(p->lchild!=NULL)
      { r=(r+1)%MaxSize;
        que[r] = p->lchild; //有左孩子, 将其入队
      }
      if(p->rchild!=NULL)
      { r=(r+1)%MaxSize;
        que[r] = p->rchild; //有右孩子, 将其入队
      }
    }
    lev++; //当前层访问完毕, 进入下一层处理
  }
  return 0; // 参数 k 超出了树的高度
}
```

2. 假设二叉树中的每个结点的值为单个字符且是唯一的，采用二叉链存储结构存储，设计一个算法 `int descendant (BTNode *b, char x, char y)`，判断值为 `x` 的结点是不是值为 `y` 的结点的子孙，若是，属于第几代子孙。算法函数返回值说明：如果结点 `x` 不是结点 `y` 的子孙结点，或者二叉树中不存在 `x` 或不存在 `y`，该算法都返回零值，否则，返回一个正整数，表示结点 `x` 属于结点 `y` 的第几代子孙。

要求：1) 先说明算法的基本设计思想，再写出用 C/C++ 语言描述的算法程序的完整代码；2) 编写程序，验证算法的正确性，提供程序测试数据和运行结果的截图。

算法 1 设计思路：首先采用递归的先序遍历方法查找值为 `y` 的结点（结点 `y`），返回指向结点 `y` 的指针；再采用递归方法求结点 `x` 在以结点 `y` 为根的子树中的层次 `k`，那么 `x` 属于 `y` 的第 `k-1` 代子孙。所以，分别定义两个递归调用的函数 `BTNode * FindNode(BTNode *b, char x)` 和 `int Level(BTNode *b, char x)` 来实现。通过调用这两个函数，可以求得二叉树 `b` 中的结点 `x` 属于结点 `y` 的第几代子孙。

对应的算法代码如下：

```
BTNode * FindNode(BTNode *b, char x) //在二叉树 b 中查找值为 x 的结点
{ BTNode *p;
  if (b==NULL) return NULL;
```

```

if(b->data == x) return b;

p=FindNode(b->lchild, x); //在左子树中查找
if(p==NULL) //在左子树中未找到, 再在右子树中查找
p=FindNode(b->rchild, x);
return p;
}

int Level(BTNode *b, char x) //求值为 x 的结点在二叉树 b 中的层次
{ int h;
  if (b==NULL) return 0;
  if(b->data == x) return 1; //x 处于根结点, 所在层次为 1.

  h=Level(b->lchild, x); //求 x 在左子树中的层次
  if(h==0) //在左子树中未找到 x, 再求 x 在右子树中的层次
    h=Level(b->rchild, x);
  return h!=0?(h+1):0;
}

int descendant(BTNode *b, char x, char y) //求二叉树 b 中的结点 x 属于结点 y 的第几代子孙
{ BTNode *p;
  int gen; ..
  p= FindNode(b,y); //在二叉树 b 中查找 y
  gen = Level(p, x); //求 x 在子二叉树 p 中的层次 return gen>0?(gen-1):0;
}

```

算法 2 设计思路: 在二叉树中找结点 y 和求结点 x 的层次, 可以用同一个递归函数来实现。采用先序遍历方法同时查找结点 x 和 y。如果先找到结点 y, 就求解结点 x 在 y 的左右子树中的层次, 若层次非零值, 它就表示 x 属于 y 的第几代, 否则, x 不在 y 的子树中, 即 x 不属于 y 的后代。如果先找到 x, 那么 x 不可能是 y 的后代, 直接返回 0。因此, 需要对二叉树中结点层次的算法进行修改。算法代码如下:

```

//下面的函数: 求二叉树 b 中结点 x 在结点 y 的左右子树中的层次。
int Levelx (BTNode *b, ElemType x, ElemType y, int h) // (h 置初值 1)
{ int k;
  if (b==NULL)
    return(0);

  else if (b->data==y && x!=y) //先找到 y
  { k = Levelx(b->lchild,x,x,1); //查找 x 在左子树中的层次 (忽略 y, h 重置初值 1)
    if (k!=0) //找到了 x
      return(k); //返回 x 在子树中的层次
    else //在左子树中未找到 x, 再在右子树中查找 x 的层次 (忽略 y, h 重置初值 1)
      return( Levelx(b->rchild,x,x,1) );
  }
  else if (b->data==x)
  { if(x!=y) //先找到 x
    return(0); //返回 0, 表示 x 不是 y 的后代
    else //后找到 x
      return(h); //返回 x 所在的层次。
  }
  else //当前结点值既不是 y 也不是 x
  { k = Levelx(b->lchild,x,y,h+1); //在左子树中查找 x 和 y
    if (k==0) //在左子树中未找到 x, 再在右子树中查找 x 和 y

```

```

k = Levelx(b->rchild,x,y,h+1);
return(k);
}
}

//下面的函数：求二叉树 b 中的结点 x 属于结点 y 的第几代子孙
int descendant(BTNode *b,ElemType x,ElemType y)
{ return( Levelx(b,x,y,1) );
}

```

第7章（树）作业：补充算法设计题

假设二叉树b采用二叉链存储结构来存储，试设计一个算法，求该二叉树中从根结点出发到叶子结点的任一条最长的路径的长度，并输出此路径上各结点的值。

二叉树的结点类型定义如下：

```

typedef struct tnode
{ ElemType data;
    (1) ;
} BTNode;

```

已知算法的部分代码如下：

```

int LongestPath( BTNode *b, ElemType longestPath[ ] )
{ if (b==NULL) return 0;
  struct
  { BTNode *node;      // 存储当前结点的指针
    int father;        // 存储父结点在队列中的位置
  }Qu[MaxSize];
  ElemType path[MaxSize];

  int front, rear, np, i;
  int pathLen, maxLen=0;
  (2) ;
  rear++;
  Qu[rear].node = b;      // 根结点指针进队
  Qu[rear].father = -1;    // 根结点没有父结点
  while( (3) )           // 队列不为空，则循环操作
  { (4) ;
    b=Qu[front].node;     // (5) (此处加注释)

    if(b->lchild != NULL)
    { (6) ;
      Qu[rear].node = b->lchild; // (7) (此处加注释)
      (8) ;
    }
    if(b->rchild != NULL)
    { (9) ;
      Qu[rear].node = b->rchild; // (10) (此处加注释)
      (11) ;
    }

    if( (12) )
    { np=front; pathLen=0;
      do // 将当前路径上的结点值转存到path[ ]数组
      { path[pathLen] = (13) ;

```

```

        pathLen++;
        np=Qu[np].father;
    }while(np != -1);
    if(      (14)      )
    {
        for(i=0; i<pathLen; i++)
            longestPath[i] =path[i];
        maxLen = pathLen;
    }
}
}
return maxLen;
}

```

答题要求:

- 在上述数字标记的位置，补充填写代码或注释。
- 说明算法中所用队列的类型和二叉树的遍历方式。
- 给出算法的时间和空间复杂度。

答: a) 代码中各空白处填写内容如下:

- ① `struct tnode *lchild, *rchild`
 - ② `front = rear = -1` 或 `front=-1; rear=-1` ③ `front != rear` 或 `front < rear`
 - ④ `front++` 或 `front=front+1`
 - ⑤ 队首元素出队
 - ⑥ `rear++` 或 `rear = rear+1`
 - ⑦ 左孩子指针入队
 - ⑧ `Qu[rear].father=front`
 - ⑨ `rear++` 或 `rear = rear+1`
 - ⑩ 右孩子指针入队
 - ⑪ `Qu[rear].father=front`
 - ⑫ `b->lchild ==NULL && b->rchild ==NULL`
 - ⑬ `Qu[np].node->data`
 - ⑭ `pathLen > maxLen`
- b) 顺序队，层次遍历。(计 2 分)
- c) 算法的时间复杂度为 $O(n^2)$
-), 空间复杂度为 $O(n)$ 。

```

typedef struct tnode
{
    ElemType data;
    struct tnode *lchild, *rchild;           // (1)
} BTreeNode;

int LongestPath( BTreeNode *b, ElemType longestPath[ ] )
{
    if (b==NULL) return 0;
    struct
    {
        BTreeNode *node;           // 存储当前结点的指针
        int father;                // 存储父结点在队列中的位置
    }Qu[MaxSize];

    ElemType path[MaxSize];

    int front, rear, np, i;
    int pathLen, maxLen=0;
    front = rear = -1;              // (2)
}

```

```

rear++;
Qu[rear].node = b;          // 根结点指针进队
Qu[rear].father = -1;       // 根结点没有父结点
while(    front != rear    ) // (3) 队列不为空，则循环操作
{
    front++;                // (4)
    b=Qu[front].node;       // (5) 队首元素出队

    if(b->lchild != NULL)
    {
        rear++;            // (6)
        Qu[rear].node = b->lchild; // (7) 左孩子指针入队
        Qu[rear].father=front;    // (8)
    }
    if(b->rchild != NULL)
    {
        rear++;            // (9)
        Qu[rear].node = b->rchild; // (10) 右孩子指针入队
        Qu[rear].father=front;    // (11)
    }

    if(    b->lchild ==NULL && b->rchild ==NULL    )
    { np=front; pathLen=0;
      do // 将当前路径上的结点值转存到path[ ]数组
      { path[pathLen] =    Qu[np].node->data    ; // (13)
        pathLen++;
        np=Qu[np].father;
      }while(np != -1);

      if(    pathLen > maxLen    )
      { for(i=0; i<pathLen; i++)
        longestPath[i] =path[i];
        maxLen = pathLen;
      }
    }
}
return maxLen;
}

```

图

邻接矩阵 矩阵具体对应权值 有向图的竖列为起点 横的为重点 无向图是对称的 有向图不对称

邻接表

广度优先搜索 (BFS)

深度优先搜索 (DFS)

拓扑排序 删除入度为0的节点

生成树 (BFS生成/DFS生成) 不保证权值最小

最小生成树 (prim算法 kruskal算法)

dijkstra算法 求一个顶点到其他顶点的最短路径

floyd算法 求两顶点之间的最短路径

- Prim 算法 (适合稠密图)
 - 一句话: 从任意起点出发, 每次选取连接“已选集合 U ”与“未选集合 $V-U$ ”的**最小权边**扩展, 直到包含所有顶点
 - 核心思想: 贪心 (局部最优逐步扩展 $\rightarrow$ 全局最优的 MST)
 - 过程要点
 - 初始化: 任选起点 s , 令 $U=\{s\}$
 - 重复 $n-1$ 次:
 - 在所有“跨越割 $(U, V-U)$ ”的边里选最小权边 (u, v)
 - 把 uv 加入 U , 并更新各未加入顶点到 U 的最小连接边 (候选边)
 - 常用实现与复杂度 (考研常考写法)
 - 邻接矩阵 + 两重循环: $O(V^2)$
 - 邻接表 + 最小堆 (优先队列): 约 $O(E \log V)$ (更工程化)
 - 为什么更适合稠密图
 - 稠密图 $E \approx V^2$:
 - Prim (矩阵) 仍是 $O(V^2)$, 不随 E 大幅变差
 - Kruskal 需要排序全部边: 约 $O(E \log E) \approx O(V^2 \log V)$, 更慢

- Kruskal 算法 (适合稀疏图)
 - 一句话: 把所有边按权值从小到大排序, 依次选取**不形成回路**的边加入, 直到选够 $n-1$ 条边
 - 核心思想: 贪心 + 判环 (避免成环)
 - 过程要点
 - 初始化: 每个顶点自成一个连通分量 (生成森林), 边集为空
 - 依次扫描已排序的边 (u, v) :
 - 若 u, v 不在同一分量 $\rightarrow$ 选入该边, 并合并两个分量
 - 否则会成环 $\rightarrow$ 丢弃
 - 直到选到 $n-1$ 条边结束
 - 实现时要解决的 3 件事 (对应课件)
 - 图怎么存: 关键是拿到**边集** (课件可用邻接矩阵枚举边; 更常用边数组/邻接表收集边)
 - 边怎么排序: 对 E 条边按权排序 (工程里用快排/堆排等)
 - 怎么判环: 用“连通分量编号/子集编号”思想; 高效实现是**并查集 (Union-Find)**
 - 复杂度
 - 主要开销在排序: $O(E \log E)$
 - 并查集合并/查找: 均摊近似线性 (常写 $O(\alpha(V))$, 考试一般忽略)
 - 为什么更适合稀疏图
 - 稀疏图 $E \ll V^2$:
 - Kruskal 复杂度 $O(E \log E) \ll O(V^2)$

- Kruskal: $O(E \log E) O(E \log E) O(E \log E)$ 很小
- Prim (矩阵): $O(V^2) O(V^2) O(V^2)$ 仍要扫大量不存在的边, 浪费明显
- 所以常见结论: 当 $E \ll V^2$ 小时, $E \log E \ll V^2 \log E \parallel V^2 E \log E \ll V^2 \rightarrow$ Kruskal 更优

• Prim vs Kruskal: 快速对比 (背诵版)

- 选边策略
 - Prim: 从一个点出发“扩展一棵树” (每次选跨割最小边)
 - Kruskal: 从最小边开始“合并森林” (每次选不成环的最小边)
- 适用场景
 - Prim: 更适合**稠密图** (矩阵实现 $O(V^2) O(V^2) O(V^2)$ 很稳)
 - Kruskal: 更适合**稀疏图** (排序边 $O(E \log E) O(E \log E) O(E \log E)$ 更省)
- 判环需求
 - Prim: 天然保持一棵树扩展, 不需要显式判环 (只连 $U \rightarrow V$ 到 $V - UV - U$)
 - Kruskal: 必须判环 (并查集)

• 易错提醒

- MST 只针对**无向图** (经典定义); 有向图对应的是其他结构 (如最小树形图等, 另题型)
- 图不连通: 不存在覆盖全部顶点的一棵 MST (只能得到最小生成森林)

• 最短路问题分类

- Dijkstra: **单源最短路 (SSSP)** —— 固定一个源点 (s), 求 (s 到) 其余所有点的最短距离/路径
- Floyd: **全源/全点对最短路 (APSP)** —— 求任意 (i 到 j) 的最短距离/路径 (得到一个距离矩阵)

• Dijkstra (狄杰斯特拉) 算法: 核心思想与步骤

- 适用图: 带权图 (**有向/无向都可**), 但要求 **所有边权非负**
- 维护集合
 - (S): 已确定最短距离的顶点集合
 - ($U = V - S$): 未确定最短距离的顶点集合
- 初始化
 - ($\text{dist}[s] = 0$), 其他 ($\text{dist} = \infty$)
 - 前驱数组 `pre[]` (或 `path[]`) 用于恢复路径
- 迭代过程
 - 从 (U) 中选取当前 (dist) 最小的顶点 (u), 加入 (S) (认为 ($\text{dist}[u]$) 最终确定)
 - 对 (u) 的所有出边做“松弛”: 若 ($\text{dist}[v] > \text{dist}[u] + w(u, v)$), 则更新 ($\text{dist}[v]$) 并更新前驱
- 结果
 - 得到从源点 (s) 到所有顶点的最短距离; 用前驱可还原最短路径树 (SPT)

• 为什么 Dijkstra 不适合负权边

- Dijkstra 的贪心前提：顶点 (u) 被选入 (S) 时，(dist[u]) 已经是最终最短距离
 - 若存在负权边，可能出现“后来通过负权边绕行使 (dist[u]) 变得更小”的情况
 - 这会破坏“入 (S) 即最终确定”的正确性 $\Rightarrow$ 可能算错
 - 若存在负权回路，最短路可能不存在（可无限变小）
- Dijkstra 能否用于带权无向图
 - **可以**，同样要求 **边权非负**
 - 无向边可视为两条反向有向边，算法逻辑不变
 - 若无向图存在负权边，往返会形成负环风险，最短路可能失去意义或算法失效
- 用 Dijkstra 的最短路边能否构成生成树（常见判断题）
 - “把所有最短路径上的边都拿出来”不一定是树（可能多于 (n-1) 条边甚至成环）
 - 只有在“为每个顶点固定选定一条最短路（确定唯一前驱）”时，前驱边集合才组成一棵 **最短路径树 SPT**
- Floyd（弗洛伊德）算法：核心思想（动态规划/递推）
 - 通常基于邻接矩阵存储，用二维数组 (A) 存最短路长度
 - (A[i][j]): 当前 (i 到 j) 的最短路长度
 - 矩阵序列含义
 - (A_k[i][j]): 只允许中间点从集合 $\{0, 1, \dots, k\}$ 中选时，(i 到 j) 的最短路长度
 - 最终 (A_{n-1}[i][j]): 任意中间点都允许时的最短路长度（全点对最短路）
 - 核心递推式
 - [

$$A_k[i][j] = \min(A_{k-1}[i][j], A_{k-1}[i][k] + A_{k-1}[k][j])$$
 - 含义：最短路要么不经过 (k)，要么经过 (k)（拆成两段）
- Floyd 如何记录/恢复路径
 - 额外用二维数组 `path`（或 `next/pre`）存路径信息
 - 当 (A[i][k]+A[k][j]) 更优时更新 `path[i][j]`（记录中转点 (k) 或记录下一跳）
 - 最终可还原任意 (i 到 j) 的最短路径
- Floyd 对边权的要求与能力
 - 允许 **负权边**（区别于 Dijkstra）
 - 若存在 **负权回路**：最短路无意义；Floyd 常用判断：若最终出现 (A[i][i]<0)，说明存在负环
- “全点对最短路”：n 次 Dijkstra vs Floyd（两者都可能写成 (O(n^3)) 的区别点）
 - 思想不同
 - Floyd：一次三重循环 DP，直接得到所有点对
 - 多次 Dijkstra：每个顶点做一次单源最短路，累计得到全点对
 - 边权限制不同
 - 多次 Dijkstra：仍要求无负权边
 - Floyd：可处理负权边（无负环）
 - 复杂度更精细的写法
 - Floyd：固定 (**O(n^3)**)

- 多次 Dijkstra：常写 ($O(n \cdot E \log V)$) (堆 + 邻接表)，稀疏图时通常更划算
- 输出形式不同
 - Floyd：天然得到距离矩阵 (配 path 可得任意点对路径)
 - 多次 Dijkstra：得到 (n) 份单源结果 (需要保存才能形成全点对)
- 选择建议 (速记)
 - 单源、无负权、图稀疏：优先 **Dijkstra (堆优化)**
 - 全点对、顶点数不大/图较稠密，或存在负权边 (无负环)：优先 **Floyd**

2. 假设一个带权有向图G采用邻接表来存储，试设计Dijkstra算法，求从源点v到其它每个顶点的最短路径和最短路径长度。

要求：在下面数字标记的空白位置，补充填写代码或注释。[提示]：填写空白位置时，可以从前后代码中找到相关线索，比如，前面的AdjGraph类型的各部分声明与后面的算法代码是相互一致的。

已知图G的存储结构类型AdjGraph的声明如下：

```
typedef struct anode
{
    int adjvex;           //邻接点编号
    int weight;           //边的权值
    (1)                   ;
} ArcNode; //边结点类型

typedef struct vnode
{
    ElemType data;        //顶点的值
    (2)                   ;
} VexNode; //顶点结点类型

typedef struct
{
    (3)                   ;
    int n,e;              //图的顶点数目和边的数目
} AdjGraph; //图的邻接表类型
```

已知算法的部分代码如下：

```
void MDijkstra (AdjGraph *G, int v) //基于邻接表的Dijkstra算法
{
    ArcNode *p;
    int dist[MAXV], path[MAXV];
    int S[MAXV];
    int minDist, i, j, u;
    for( i=0; i < G->n; i++) //初始化各数组
    {
        dist[i] = INF;      // INF为预定义的符号常量，表示无穷大
        path[i] = -1;
        (4)                 ;
    }
    S[v] = 1;              //将源点v加入集合S中
    p = G->adjlist[v].firstarc;
    while(p!=NULL)
    {
        (5)                 ; // (6) (此处加注释)
        path[p->adjvex] = v; //到邻接点的前一个顶点是源点v
        p = p->nextarc;      // (7) (此处加注释)
    }
    for(i=0; i < G->n - 1; i++) //循环，直到所有顶点的最短路径都求出
```

```

{ minDist = INF;
  For(j=0; j < G->n; j++) // (8) (此处加注释)
    if ( S[j]==0 && dist[j] < minDist )
    { u = j;
      (9) ;
    }
    (10) ; // (11) (此处加注释)
    p = (12) ;
    while (p!=NULL)
    { j = p->adjvex;
      if( S[j] ==0 && (13) )
      { (14) ; //修改不在s中的顶点的最短路径长度
        (15) ;
      }
      p = p->nextarc;
    }
  }
  Dispath(G, dist, path, S, v); //输出所有最短路径信息
}

```

① struct anode *nextarc;
 ② ArcNode *firstarc;
 ③ VexNode adjlist[MAXV];
 ④ S[i] = 0
 ⑤ dist[p->adjvex] = p->weight
 ⑥ 源点到邻接点的距离就是边的权值
 ⑦ 移动指针 p，使它指向下一个邻接点
 ⑧ 从尚未求得最短路径的顶点集（不在 S 集）中寻找距离最小的顶点
 ⑨ minDist = dist[j]
 ⑩ S[u] = 1
 ⑪ 将距离最小(路径最短)的顶点 u 加入 S 集合
 ⑫ G->adjlist[u].firstarc
 ⑬ dist[u] + p->weight < dist[j]
 ⑭ dist[j] = dist[u] + p->weight
 ⑮ path[j] = u

```

typedef struct anode
{ int adjvex; //邻接点编号
  int weight; //边的权值
  struct anode *nextarc; // (1)
} ArcNode; //边结点类型

```

```

typedef struct vnode
{ ElemType data; //顶点的值
  ArcNode *firstarc; // (2)
} VexNode; //顶点结点类型

```

```

typedef struct
{ VexNode adjlist[MAXV]; // (3)
  int n,e; //图的顶点数目和边的数目
} AdjGraph; //图的邻接表类型

```

```

void MDijkstra (AdjGraph *G, int v) //基于邻接表的Dijkstra算法

```

```

{   ArcNode *p;
    int dist[MAXV], path[MAXV];
    int S[MAXV];
    int minDist, i, j, u;

    for( i=0; i < G->n; i++) //初始化各数组
    {   dist[i] = INF;        // INF为预定义的符号常量，表示无穷大
        path[i] = -1;
        S[i] = 0;            // (4)
    }

    S[v] = 1;                //将源点v加入集合S中
    p = G->adjlist[v].firstarc;
    while(p!=NULL)
    {   dist[p->adjvex] = p->weight;    // (5)  // (6) 源点到邻接点的距离就是边的权
        path[p->adjvex] = v; //到邻接点的前一个顶点是源点v
        p = p->nextarc;    // (7) 移动指针 p，使它指向下一个邻接点
    }

    for(i=0; i < G->n - 1; i++) //循环，直到所有顶点的最短路径都求出
    {   minDist = INF;
        For(j=0; j < G->n; j++) // (8) 从尚未求得最短路径的顶点集（不在 S 集）中寻找距离
            最小的顶点
            if ( S[j]==0 && dist[j] < minDist )
            {   u = j;
                minDist = dist[j];    // (9)
            }

            S[u] = 1;                // (10)  // (11) 将距离最小(路径最短)的顶点
            u 加入 S 集合
            p = G->adjlist[u].firstarc;    // (12)

            while (p!=NULL)
            {   j = p->adjvex;
                if( S[j] ==0 && dist[u] + p->weight < dist[j] ) // (13)
                {   dist[j] = dist[u] + p->weight;    // (14) 修改不在S中的
                    顶点的最短路径长度
                    path[j] = u;    // (15)
                }
                p = p->nextarc;
            }
        }

        Dispath(G, dist, path, S, v);    //输出所有最短路径信息
    }
}

```

```
#include <stdio.h>
```

```
#define MAXV 100
```

```

#define INF 32767 // 课件里用 INF 表示无穷大

typedef struct
{
    int n, e; // 顶点数 n、边数 e (e 可不严格用到)
    int edges[MAXV][MAXV]; // 邻接矩阵: 无边=INF, 对角线=0
} MatGraph;

/* 初始化图: 设置 n; 把无边置 INF; 对角线置 0 */
void InitGraph(MatGraph *g, int n)
{
    int i, j;
    g->n = n;
    g->e = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            g->edges[i][j] = (i == j) ? 0 : INF;
}

/* 添加无向边 (u,v) 权 w: 邻接矩阵两边都要填 */
void AddEdge(MatGraph *g, int u, int v, int w)
{
    g->edges[u][v] = w;
    g->edges[v][u] = w;
    g->e++;
}

/*
 * Prim(g, v): 从起点 v 开始构造最小生成树
 * 输出内容:
 * - 每次加入一条 MST 边: (closest[k], k) 及其权值
 * - 最终输出 MST 总权值 sum
 */
void Prim(MatGraph g, int v)
{
    int lowcost[MAXV]; // lowcost[j]: j 到集合 U 的最小边权
    int closest[MAXV]; // closest[j]: 与 j 相连的 U 中顶点
    int i, j, k; // k: 每轮选出的“离 U 最近”的顶点
    int min; // 每轮的最小边权
    int sum = 0; // MST 总权值

    /* ----- 1) 初始化: U 只有一个起点 v ----- */
    /*
     * 对所有顶点 i:
     * lowcost[i] = 边 (v,i) 的权值 (若无边则 INF)
     * closest[i] = v (表示目前认为 v 是离 i 最近的 U 中顶点)
     */
    for (i = 0; i < g.n; i++)
    {
        lowcost[i] = g.edges[v][i];
        closest[i] = v;
    }

    /*
     * 用 lowcost[v] = 0 表示 v 已经加入 U
     * (课件同款技巧: lowcost==0 => 在U中)
     */
}

```

```

    */
    lowcost[v] = 0;

    /* ----- 2) 还需要选择 (n-1) 条边, 加入 (n-1) 个顶点 ----- */
    for (i = 1; i < g.n; i++)    // 从第 1 次到第 n-1 次
    {
        min = INF;
        k = -1;

        /* 2.1 在 v-u 中找 lowcost 最小的顶点 k (即候选边中最小边) */
        for (j = 0; j < g.n; j++)
        {
            /*
             * lowcost[j] != 0 说明 j 还不在于 u 中
             * lowcost[j] < min 说明找到更短的“j 到 u 的最小边”
             */
            if (lowcost[j] != 0 && lowcost[j] < min)
            {
                min = lowcost[j];
                k = j;
            }
        }

        /* 若 k 仍为 -1 或 min 为 INF, 说明图不连通, 无法生成 MST */
        if (k == -1 || min == INF)
        {
            printf("图不连通, 无法生成最小生成树 (从顶点 %d 出发无法覆盖所有顶点)。\\n",
v);

            return;
        }

        /* 2.2 输出本轮选择的最小边, 并把顶点 k 加入 u */
        /*
         * closest[k] 是 u 中与 k 相连的顶点
         * (closest[k], k) 就是本轮加入 MST 的边
         */
        printf("边(%d, %d) 权为:%d\\n", closest[k], k, min);

        sum += min;          // 累加 MST 权值
        lowcost[k] = 0;      // 标记 k 已加入 u (课件做法)

        /* 2.3 更新: k 加入 u 后, 可能会让某些顶点 j 到 u 的最短连接边变得更小 */
        for (j = 0; j < g.n; j++)
        {
            /*
             * 只考虑 v-u 中的顶点 (lowcost[j] != 0)
             * 若边 (k, j) 比当前记录的 lowcost[j] 更小, 则更新:
             *   lowcost[j] = g.edges[k][j]
             *   closest[j] = k (说明 j 现在离 k 最近)
             */
            if (lowcost[j] != 0 && g.edges[k][j] < lowcost[j])
            {
                lowcost[j] = g.edges[k][j];
                closest[j] = k;
            }
        }
    }
}

```

```

}

printf("最小生成树总权值:%d\n", sum);
}

```

查找

- 查找 (Search) 的基本概念
 - 查找表: 由若干记录 (关键字 key) 组成的数据集合
 - 查找结果: 成功 (找到 key 对应记录) / 失败 (不存在该 key)
 - 衡量查找算法性能的主要指标: **平均查找长度 ASL (Average Search Length)**
 - 本质: 关键字比较次数的期望值
 - 常分: $(ASL_{\text{成功}})$ 、 $(ASL_{\text{失败}})$
- ASL 的一般表达式
 - $(ASL = \sum p_i \cdot c_i)$
 - (p_i) : 第 (i) 种情况发生的概率 (成功时为查到某个记录的概率; 失败时为落入某个区间/外部结点的概率)
 - (c_i) : 该情况下需要的关键字比较次数
- 顺序查找 (线性查找)
 - 思想: 从表头到表尾逐个比较
 - 成功情况
 - 查到第 (i) 个元素需要比较 (i) 次
 - 若等概率 $(p_i = \frac{1}{n})$:
 - $(ASL_{\text{成功}} = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2})$
 - 直观: 平均比较次数约为表长一半
 - 失败情况
 - 需要与表中所有元素都比较一次
 - $(ASL_{\text{失败}} = n)$
 - 时间复杂度: $(O(n))$
 - 特点
 - 优点: 不要求有序、实现简单
 - 缺点: 表长大时效率低
- 折半查找 (二分查找) 前提与思想
 - 前提: 线性表必须**有序** (通常是顺序存储数组)
 - 思想: 每次取中点 (mid) 比较, 把区间缩小为原来的一半
 - 为什么效率高: 规模递减 $(n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots)$, 比较次数与 $(\log_2 n)$ 同阶
 - 递归实现要点
 - 递归区间 $([low, high])$

- 若 $(low > high)$ 则失败
 - $(mid = \lfloor (low + high) / 2 \rfloor)$
 - key 与 $A[mid]$ 比较后递归左半或右半
- 折半查找的判定树（决策树）理解（你课件里的重点）
 - 内部结点：关键字（成功时命中某个内部结点）
 - 外部结点：不在表中的“区间”（失败时落入某个外部结点）
 - 比较次数的含义
 - 成功：从根到目标内部结点路径上经历的比较次数 = 该结点层数
 - 失败：从根到某个外部结点路径上经历的比较次数 = 路径上内部结点个数（层数）
- 折半查找的 ASL（结合课件结论）
 - 成功 ASL（等概率假设）
 - 判定树高度 $(h = \log_2(n+1))$
 - $(ASL_{\text{成功}} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1)$
 - 失败 ASL（按外部结点做平均）
 - 失败时会落到某个外部结点，对所有外部结点的比较次数做平均
 - 你给的示例：外部结点 12 个，其中 4 个需要比较 3 次、8 个需要比较 4 次
 - $(ASL_{\text{失败}} = \frac{4 \times 3 + 8 \times 4}{12} = 3.67)$
 - 示例查 26：比较 $25 \rightarrow 30 \rightarrow 28$ ，共 3 次，落在区间 25~28（外部结点）
- 折半查找的最坏比较次数
 - 对 (n) 个元素：
 - 成功最坏比较次数： $(\lceil \log_2(n+1) \rceil)$
 - 失败最坏比较次数： $(\lceil \log_2(n+1) \rceil)$
- 折半查找时间复杂度
 - 递推： $(T(n) = T(n/2) + O(1))$
 - 结论： $(T(n) = O(\log_2 n))$
- 顺序查找 vs 二分查找（速记对比）
 - 顺序查找：不要求有序， $(O(n))$ ；等概率下 $(ASL_{\text{成功}} = \frac{n+1}{2})$ ， $(ASL_{\text{失败}} = n)$
 - 二分查找：要求有序（多用于数组）， $(O(\log n))$ ；比较次数与判定树高度（ $(\log n)$ 量级）相关

补充思考题：

1. 顺序查找法、二分查找法和分块查找法对被查找表的存储结构和数据元素的排列方式分别有什么要求？

(1) 顺序查找法

存储结构要求：无特殊要求，既适用于顺序存储结构，也适用于链式存储结构。

数据元素排列方式要求：不要求数据元素按关键字有序排列，表中元素可以是无序的。

(2) 二分查找法（折半查找法）

存储结构要求：必须采用顺序存储结构，要求能够按下标进行随机访问。

数据元素排列方式要求：表中数据元素必须按关键字有序排列（通常为递增或递减有序）。

(3) 分块查找法（索引顺序查找法）

存储结构要求：需要建立索引表和主表，主表可以是顺序存储或链式存储，索引表一般采用顺序存储。

数据元素排列方式要求：块间必须按关键字有序排列；块内数据元素可以是无序的。

2. B树和B+树有何区别？它们跟AVL树有何相同和不同之处？

(1) B树与B+树的区别：

B树：关键字和记录可以同时存放在各级结点中，查找可能在非叶子结点结束；叶子结点之间一般没有链式连接。

B+树：非叶子结点只存放关键字作为索引，所有记录都存放在叶子结点中；叶子结点按关键字大小用指针连接成有序链表，便于顺序和范围查找。

(2) B树、B+树与AVL树的相同点：

都是平衡查找树；

都保持关键字的有序性；

查找、插入和删除的时间复杂度均为 $O(\log n)$ 。

(3) B树、B+树与AVL树的不同点：

AVL树是二叉平衡查找树，结点最多有两个孩子，通过旋转操作保持严格平衡；

B树和B+树是多路平衡查找树，结点可有多个孩子，通过结点分裂和合并保持平衡；

B树和B+树树高较低，更适合外存和数据库索引，而AVL树多用于内存结构。

3. 哈希查找方法的时间复杂度为多少？采用哈希查找法的前提条件是什么？

(1) 哈希查找方法的时间复杂度：

在理想情况下，哈希查找的时间复杂度为 $O(1)$ ；

在发生较多冲突时，最坏情况下时间复杂度为 $O(n)$ 。

(2) 采用哈希查找法的前提条件：

必须构造合适的哈希函数，使关键字尽量均匀地分布到哈希表中；需要确定并采用有效的冲突处理方法（如链地址法）；哈希表需要有足够的存储空间，装填因子不能过大，以保证较高的查找效率。

*4. 什么是哈希冲突？解决哈希冲突有哪些方法？

哈希冲突是指：不同的关键字经过同一个哈希函数计算后，得到相同的哈希地址，从而需要存放到同一位置或同一桶中的现象。

解决哈希冲突的常用方法：

开放定址法：当发生冲突时，按一定规则寻找下一个空的存储位置

链地址法：将哈希地址相同的记录用链表连接存放；

再哈希法：发生冲突时，使用另一个哈希函数重新计算地址；

建立公共溢出区：将发生冲突的记录统一存放在溢出区中。

```
// 快速排序：对 R[s] 到 R[t] 区间进行排序
void QuickSort(RecType R[], int s, int t)
{
    int i = s, j = t;    // i: 从左向右扫描指针；j: 从右向左扫描指针
    RecType tmp;         // tmp: 暂存基准记录 (pivot)

    // 递归出口：当区间内少于 2 个元素 (s >= t) 时，不需要排序
    if (s < t)
    {
        tmp = R[s];      // 选择区间第 1 个元素作为基准 (pivot)，先“挖坑”出来

        // 一次划分：通过 i、j 两端向中间扫描，把小于 pivot 的放左边，大于 pivot 的放右边
        while (i != j)
        {
            // 1) 从右侧开始找：寻找第一个 key < pivot.key 的元素
            //    条件 R[j].key >= tmp.key 的都应该放在右边，所以继续 j--
            while (j > i && R[j].key >= tmp.key)
                j--;

            // 找到了一个应放到左边的元素 R[j]，把它填到左边的“坑” R[i]
            // 此时 j 位置变成新的“坑”
            R[i] = R[j];

            // 2) 从左侧开始找：寻找第一个 key > pivot.key 的元素
            //    条件 R[i].key <= tmp.key 的都应该放在左边，所以继续 i++
            while (i < j && R[i].key <= tmp.key)
                i++;

            // 找到了一个应放到右边的元素 R[i]，把它填到右边的“坑” R[j]
            // 此时 i 位置变成新的“坑”
            R[j] = R[i];
        }

        // i == j 时，坑位最终落在 i (或 j) 处，把基准元素放回去
        R[i] = tmp;
    }
}
```

```

// 划分完成: R[s..i-1] <= pivot, R[i] == pivot, R[i+1..t] >= pivot
R[i] = tmp;

// 递归排序基准左边子区间
QuickSort(R, s, i - 1);

// 递归排序基准右边子区间
QuickSort(R, i + 1, t);
}
}

```

排序

排列方法	$T(n)$		$S(n)$		稳定性
	平均时间	最坏时间	平均	最坏	
直接/折半插入	$O(n^2)$		$O(1)$		稳定
希尔排序	与所取增量序列有关		$O(1)$		不稳定
简单选择	$O(n^2)$		$O(1)$		不稳定
冒泡排序	$O(n^2)$		$O(1)$		稳定
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(\log_2 n)$	$O(n)$	不稳定
堆排序	$O(n \log_2 n)$		$O(1)$		不稳定
归并排序	$O(n \log_2 n)$		$O(n)$		稳定
基数排序	$O(d(n + rd))$		$O(n + rd)$		稳定

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n \log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$			$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定

9. 线性表有顺序表和链表两种存储方式，不同的排序方法适合不同的存储结构。对于常见的内部排序方法，说明哪些更适合于顺序表？哪些更适合于链表？哪些两者都适合？

更适合于顺序表的排序方法有希尔排序、折半插入排序、快速排序、堆排序和归并排序。

更适合于链表的排序方法是基数排序。

两者都适合的排序方法有直接插入排序、冒泡排序和简单选择排序。

补充作业题：

【题1】：某关键字序列R为 (5,1,9,2,7,6,4,8,0,3) ,用下列各排序方法将R中的元素递增排序。

- (1) 分别给出希尔排序和二路归并排序的第一趟和第二趟排序后的关键字序列。
- (2) 分别给出冒泡排序、直接插入排序和简单选择排序的第一趟和第二趟排序后的关键字序列。
- (3) 取区间第一个元素作为划分基准，给出快速排序的第一趟和第二趟排序后的关键字序列。
- (4) 给出将R调整成初始堆后的关键字序列，以及第一趟和第二趟堆排序后的关键字序列。

(1) 希尔排序、二路归并排序

① 希尔排序

取增量序列：5, 2, 1

第一趟 (d = 5)

分组并做直接插入排序：

(5,6) → (5,6)

(1,4) → (1,4)

(9,8) → (8,9)

(2,0) → (0,2)

(7,3) → (3,7)

合并后序列：

(5, 1, 8, 0, 3, 6, 4, 9, 2, 7)

第二趟 (d = 2)

对子序列分别插入排序后得到：

(3, 0, 2, 1, 4, 6, 5, 7, 8, 9)

② 二路归并排序

第一趟 (子序列长度 = 1)

两两归并：

(1, 5, 2, 9, 6, 7, 4, 8, 0, 3)

第二趟 (子序列长度 = 2)

继续归并：

(1, 2, 5, 9, 4, 6, 7, 8, 0, 3)

(2) 冒泡排序、直接插入排序、简单选择排序

① 冒泡排序

第一趟

最大元素 9 冒到最后：

(1, 5, 2, 7, 6, 4, 8, 0, 3, 9)

第二趟

次大元素 8 冒到倒数第二：

(1, 2, 5, 6, 4, 7, 0, 3, 8, 9)

② 直接插入排序

第一趟（插入 1）

(1, 5, 9, 2, 7, 6, 4, 8, 0, 3)

第二趟（插入 9）

(1, 5, 9, 2, 7, 6, 4, 8, 0, 3)

③ 简单选择排序

第一趟（选最小 0）

(0, 1, 9, 2, 7, 6, 4, 8, 5, 3)

第二趟（选次小 1）

(0, 1, 9, 2, 7, 6, 4, 8, 5, 3)

(3) 快速排序

取区间第一个元素为基准

第一趟（基准 = 5）

划分结果：

(3, 1, 0, 2, 4, 5, 7, 6, 8, 9)

第二趟

分别对左右子区间进行一次划分：

左区间（3 为基准）→ 不再交换

右区间（7 为基准）→ 不再交换

结果：

$(2, 1, 0, 3, 4, 5, 6, 7, 8, 9)$

(4) 堆排序

① 初始堆

调整为小根堆后：

$(0, 1, 4, 2, 3, 6, 9, 8, 5, 7)$

② 第一趟堆排序

交换堆顶与末尾，再调整：

$(1, 2, 4, 5, 3, 6, 9, 8, 7, 0)$

③ 第二趟堆排序

再次交换并调整：

$(2, 3, 4, 5, 7, 6, 9, 8, 1, 0)$

【题2】：关于堆排序方法，为什么使用大根堆进行递增排序时，空间复杂度为 $O(1)$ ，而使用小根堆进行递增排序时，空间复杂度为 $O(n)$ ？如果要设计一个空间复杂度为 $O(1)$ 的递减排序算法，应该使用大根堆还是小根堆？

使用大根堆进行递增排序，大根堆的堆顶是当前最大元素

每一趟排序：将堆顶（最大值）与当前序列末尾元素交换，调整剩余元素为大根堆，排好序的最大元素依次放在数组尾部，全过程都在原数组中进行，不需要额外辅助存储空间，空间复杂度为 **$O(1)$**

使用小根堆进行递增排序，小根堆的堆顶是当前最小元素

每一趟取出堆顶元素后：，最小元素应该放在有序序列的前端，但原数组前部仍需保存尚未排序的元素，因此通常需要另设一个辅助数组，按顺序存放依次取出的最小元素，最终再形成递增序列，需要额外长度为 **n** 的存储空间，空间复杂度为 **$O(n)$**

设计空间复杂度为 $O(1)$ 的递减排序算法，递减排序要求：每一趟确定一个最小元素，并将其放到序列末尾，应使用小根堆

原因：小根堆堆顶是最小元素，每次将堆顶与当前末尾元素交换，排好序的最小元素依次放在数组尾部，不需要额外辅助空间。

【题3】：对冒泡排序算法进行改进，减少每趟排序时元素交换赋值的次数，并做实验验证。比如，对9~0十个整数的递减序列进行递增排序，分别用常规冒牌排序算法和改进算法进行排序，总的赋值操作次数分别是多少？编程输出结果。

```
#include <stdio.h>
```

```
#define N 10
```

```
int bubble(int a[]) {
```

```
    int i, j, t;
```

```
    int assign = 0;
```

```
    for (i = 0; i < N - 1; i++) {
```

```
        for (j = 0; j < N - 1 - i; j++) {
```

```
            if (a[j] > a[j + 1]) {
```

```
                t = a[j];  assign++;
```

```
                a[j] = a[j + 1]; assign++;
```

```
                a[j + 1] = t; assign++;
```

```
            }
```

```
        }
```

```
    }
```

```
    return assign;
```

```
}
```

```
int bubble_improve(int a[]) {
```

```
    int i, j, temp;
```

```
    int assign = 0;
```

```
    for (i = 0; i < N - 1; i++) {
```

```
        temp = a[i + 1]; assign++;
```

```
        j = i;
```

```
        while (j >= 0 && a[j] > temp) {
```

```
    a[j + 1] = a[j]; assign++;  
    j--;  
}  
a[j + 1] = temp; assign++;  
}  
return assign;  
}
```

```
int main() {  
    int a1[N] = {9,8,7,6,5,4,3,2,1,0};  
    int a2[N] = {9,8,7,6,5,4,3,2,1,0};  
  
    printf("常规冒泡排序赋值次数: %d\n", bubble(a1));  
    printf("改进冒泡排序赋值次数: %d\n", bubble_improve(a2));  
  
    return 0;  
}
```

常规冒泡排序赋值次数: 135

改进冒泡排序赋值次数: 63